

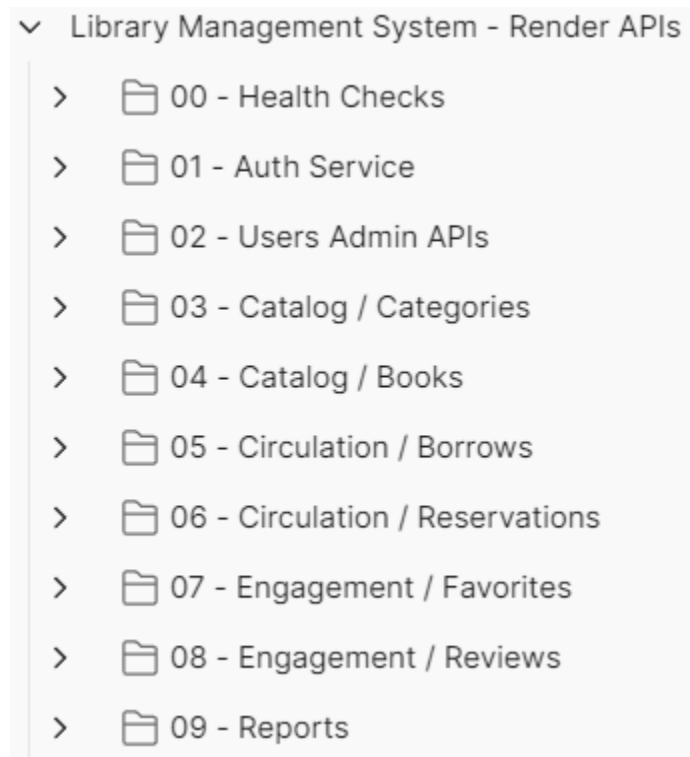
Library Management System – Evidence & Requirements Documentation

1. Implementation (APIs)

API Collection Structure

The following screenshot shows the organized Postman collection used for testing all system APIs across all microservices.

- Health Checks
- Authentication Service
- Users / Admin APIs
- Catalog APIs
- Circulation APIs
- Engagement APIs
- Reports APIs



API Collection Structure

Authentication – Login API

Endpoint

POST /api/auth/login

Features Demonstrated

- JWT Authentication
- Secure Login
- Role-based Authorization
- Token Generation

The screenshot shows a REST client interface with the following details:

- URL:** `{{baseUri}}/api/auth/login`
- Method:** POST
- Body (raw):**

```
1 {
2   "email": "admin@library.com",
3   "password": "Admin@123456"
4 }
```
- Status:** 200 OK
- Response (JSON):**

```
2  "success": true,
3  "message": "User logged in successfully",
4  "data": {
5    "accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbnRg2MmY2YS1lbnJmLTQ5ZWYtYWEyYS94ZTE3MD1jYW5kZDEiLCJlbWVpbCI6ImFkbWluQGxpYnJhcnkuY29tIiwicm9sZSI6IkFETU10IiwiaWF0Ij0xNzc4Mzc1OTU2LCJleHAiOjE3Nzg0NjIzNTZ9.gQWcuXAV-8e-U-MPDE7TbFbo99TWd0_dirSrtgKqcn2o",
6    "tokenType": "Bearer",
7    "expiresInMinutes": 1440,
8    "user": {
9      "id": "45862f6a-e630-49ef-aa2a-8e1709caa9d1",
10     "fullName": "System Admin",
11     "email": "admin@library.com",
12     "phone": "01000000000",
13     "role": "ADMIN",
14     "active": true,
15     "createdAt": "2026-05-09T20:01:15.691328Z",
16     "updatedAt": "2026-05-09T20:01:15.691328Z"
17   }
18 }
```

Login API

Users Administration – Get Users

Endpoint

GET /api/users

Features Demonstrated

- Admin Authorization
- Pagination

- Secure Protected Endpoints

Library Management System - Render APIs / 02 - Users Admin APIs / **Get Users**

GET `{{baseUrl}}/api/users?page=0&size=10&sort=createdAt` **Send**

Docs Params Authorization Headers (8) **Body** Scripts Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies Headers (22) Test Results **200 OK** • 623 ms • 1.13 KB • Save Response

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Users retrieved successfully",
4   "data": {
5     "content": [
6       {
7         "id": "8236eb27-4c7a-4407-b8df-4dcf2df20673",
8         "fullName": "kareem",
9         "email": "kareem@gmail.com",
10        "phone": "01004385166",
11        "role": "USER",
12        "active": true,
13        "createdAt": "2026-05-09T19:16:59.372697Z",
14        "updatedAt": "2026-05-09T22:35:08.086956Z"
15      },
16      {
17        "id": "45862f6a-e630-49ef-aa2a-8e1709caa9d1",
18        "fullName": "System Admin",
19        "email": "admin@library.com",
20        "phone": "01000000000",
21        "role": "ADMIN",
22        "active": true,
23        "createdAt": "2026-05-09T20:01:15.691328Z",
24        "updatedAt": "2026-05-09T20:01:15.691328Z"
25      }
26    ]
27   }
28 }
```

Get Users

Users Administration – Activate User

Endpoint

PATCH `/api/users/{id}/activate`

Features Demonstrated

- Role-Based Access Control
- Account Management
- Update Operations

Library Management System - Render APIs / 02 - Users Admin APIs / **Activate User**

Save

Share

PATCH

{{baseUri}} /api/users/{{userId}} /activate

Send

Docs

Params

Authorization

Headers (9)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description		Bulk Edit
	Key	Value	Description		

Body

Cookies

Headers (22)

Test Results

200 OK

724 ms

906 B

Save Response

{ } JSON

Preview

Visualize

```
1 {
2   "success": true,
3   "message": "User activated successfully",
4   "data": {
5     "id": "45862f6a-e630-49ef-aa2a-8e1709caa9d1",
6     "fullName": "System Admin",
7     "email": "admin@library.com",
8     "phone": "01000000000",
9     "role": "ADMIN",
10    "active": true,
11    "createdAt": "2026-05-09T20:01:15.691328Z",
12    "updatedAt": "2026-05-09T20:01:15.691328Z"
13  },
14   "timestamp": "2026-05-10T01:20:56.478813375Z"
15 }
```

Activate User

Catalog Service – Create Category

Endpoint

POST /api/categories

Features Demonstrated

- CRUD Operations
- Validation
- Catalog Management

The screenshot displays a REST client interface. At the top, a POST request is configured to the endpoint `{{baseUrl}}/api/categories`. The request body is a JSON object: `{ "name": "Computer Science 1", "description": "Books related to CS" }`. Below the request, the response is shown as a 201 Created status with a response time of 419 ms and a size of 870 B. The response body is a JSON object: `{ "success": true, "message": "Category created successfully", "data": { "id": "c4ae00e1-e2b3-4446-a3dc-e62ed749d910", "name": "Computer Science 1", "description": "Books related to CS", "createdAt": "2026-05-10T01:22:34.879872177Z", "updatedAt": "2026-05-10T01:22:34.879872177Z" }, "timestamp": "2026-05-10T01:22:34.886982685Z" }`.

Create Category

Catalog Service – Get Categories

Endpoint

GET `/api/categories`

Features Demonstrated

- Data Retrieval
- API Pagination Support
- Secure Service Communication

Library Management System - Render APIs / 03 - Catalog / Categories / Get All Categories

GET `{{baseUrl}}/api/categories` Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
-----	-------	-------------	-----------

Body Cookies Headers (22) Test Results 200 OK • 260 ms • 1.23 KB Save Response

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Categories retrieved successfully",
4   "data": [
5     {
6       "id": "11111111-1111-1111-1111-111111111111",
7       "name": "Programming",
8       "description": "Books about programming and software development",
9       "createdAt": "2026-05-09T19:04:07.543769Z",
10      "updatedAt": "2026-05-09T19:04:07.543769Z"
11    },
12    {
13      "id": "22222222-2222-2222-2222-222222222222",
14      "name": "Database",
15      "description": "Books about database systems and SQL",
16      "createdAt": "2026-05-09T19:04:07.543769Z",
17      "updatedAt": "2026-05-09T19:04:07.543769Z"
18    },
19    {
20      "id": "33333333-3333-3333-3333-333333333333",
21      "name": "Software Engineering",
22      "description": "Books about software engineering principles and practices",
23      "createdAt": "2026-05-09T19:04:07.543769Z",
24      "updatedAt": "2026-05-09T19:04:07.543769Z"
25    }
26  ]
27 }
```

Get Categories

Catalog Service – Create Book

Endpoint

POST /api/books

Features Demonstrated

- DTO Usage
- Relationship Mapping
- Business Validation

Library Management System - Render APIs / 04 - Catalog / Books / **Create Book**

POST `{{baseUri}}/api/books` **Send**

Docs Params Authorization Headers (11) **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON [Schema](#) [Beautify](#)

```
1 {
2   "title": "Code",
3   "author": "Robert C. Martin",
4   "isbn": "9780192350884",
5   "categoryId": "{{categoryId}}",
6   "publisher": "Prentice Hall",
7   "publicationYear": 2008,
8   "description": "A handbook of agile software craftsmanship.",
9   "totalCopies": 5,
```

Body Cookies Headers (22) Test Results 201 Created • 427 ms • 1017 B Save Response

{ JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Book created successfully",
4   "data": {
5     "id": "40b02641-b030-4c77-a144-481ec0da7242",
6     "title": "Code",
7     "author": "Robert C. Martin",
8     "isbn": "9780192350884",
9     "description": "A handbook of agile software craftsmanship.",
10    "coverImageUrl": null,
11    "categoryId": "f4c3a169-463f-4f50-aea6-161b2a0eeb67",
12    "categoryName": "Software Engineering-2",
13    "totalCopies": 5,
14    "availableCopies": 5,
15    "available": true,
16    "publishedYear": null,
17    "createdAt": "2026-05-10T01:23:31.869030325Z",
```

Create Book

Catalog Service – Search Books

Endpoint

GET `/api/books`

Features Demonstrated

- Filtering
- Search Functionality
- Availability Checks

Library Management System - Render APIs / 04 - Catalog / Books / Search Books

GET `{{baseUri}}/api/books?title=&author=&isbn=&categoryId=&availableOnly=false&page=0&size=10` Send

Auth Type: Be... Token: `{{token}}`

The authorization header will be automatically generated when you send the

Body Cookies Headers (22) Test Results 200 OK • 266 ms • 1.59 KB Save Response

JSON Preview Visualize

```
6 {
7   "id": "bbbbbbb-bbbb-bbbb-bbbb-bbbb-bbbb-bbbb",
8   "title": "Database System Concepts",
9   "author": "Abraham Silberschatz",
10  "isbn": "978073523323",
11  "description": "A classic textbook about database systems",
12  "coverImageUrl": "https://res.cloudinary.com/demo/image/upload/sample.jpg",
13  "categoryId": "2222222-2222-2222-2222-222222222222",
14  "categoryName": "Database",
15  "totalCopies": 4,
16  "availableCopies": 4,
17  "available": true,
18  "publishedYear": 2010,
19  "createdAt": "2026-05-09T19:04:07.543769Z",
20  "updatedAt": "2026-05-09T19:04:07.543769Z"
21 },
22 {
23   "id": "06b65743-cfc5-43a0-97be-e69ab2c34b24",
24   "title": "Clean Code",
25   "author": "Robert C. Martin",
26   "isbn": "9780132350844",
27   "description": "A handbook of agile software craftsmanship."
```

Search Books

Circulation Service – Borrow Request

Endpoint

POST `/api/borrows`

Features Demonstrated

- Borrow Workflow
- Book Availability Logic
- Request Status Handling

Library Management System - Render APIs / 05 - Circulation / Borrows / Request Borrow

POST `{{baseUrl}}/api/borrows` Save Share Send

Docs Params Authorization Headers (11) **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   "bookId": "{{bookId}}"
3 }
```

Body Cookies Headers (22) Test Results 201 Created 406 ms 942 B Save Response

{} JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Borrow request created successfully",
4   "data": {
5     "id": "2753b3e5-7e45-47b1-8a11-7863826a4517",
6     "userId": "8236eb27-4c7a-4407-b8df-4dcf2df20673",
7     "bookId": "aaaaaaaa-aaaa-aaaa-aaaaaaaaaaaa",
8     "status": "PENDING",
9     "borrowDate": null,
10    "dueDate": null,
11    "returnedDate": null,
12    "fineAmount": 0,
13    "approvedBy": null,
14    "createdAt": "2026-05-10T02:16:50.688256234Z",
15    "updatedAt": "2026-05-10T02:16:50.688256234Z"
16  },
17   "timestamp": "2026-05-10T02:16:50.699967938Z"
18 }
```

Borrow Request

Users Administration – Update Role

Endpoint

PATCH `/api/users/{id}/role`

Features Demonstrated

- RBAC Authorization
- Dynamic Role Management

PATCH `{{baseUri}} /api/users/ {{userid}} /role` Send

Docs Params Authorization Headers (11) **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema Beautify

```
1 {
2   |   "role": "LIBRARIAN"
3 }
```

Body Cookies Headers (22) Test Results 200 OK • 552 ms • 895 B Save Response

JSON Preview Visualize

```
1 {
2   |   "success": true,
3   |   "message": "User role updated successfully",
4   |   "data": {
5   |     |   "id": "794132db-b764-4327-ae20-2163fiedfd06",
6   |     |   "fullName": "User1",
7   |     |   "email": "User1@gmail.com",
8   |     |   "phone": "01000000001",
9   |     |   "role": "LIBRARIAN",
10  |     |   "active": true,
11  |     |   "createdAt": "2026-05-09T21:05:53.130515Z",
12  |     |   "updatedAt": "2026-05-09T21:05:53.130515Z"
13  |     | }
14  |   },
15  |   "timestamp": "2026-05-10T02:20:52.970571204Z"
16 }
```

Update Role

Engagement Service – Add Favorite

Endpoint

POST /api/favorites

Features Demonstrated

- User Interaction
- Personalized User Features

Library Management System - Render APIs / 07 - Engagement / Favorites / Add Favorite

POST {{baseUrl}}/api/favorites

Send

Docs Params Authorization Headers (11) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1 {
2   "bookId": "{{bookId}}"
3 }
```

Body Cookies Headers (22) Test Results

201 Created • 832 ms • 876 B Save Response

{ JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Favorite added successfully",
4   "data": {
5     "id": "e928a1e5-e3e7-4747-850d-cfeba66c23e6",
6     "userId": "8236eb27-4c7a-4407-b8df-4dcf2df20673",
7     "bookId": "aaaaaaaa-aaaa-aaaa-aaaaaaaaaaaa",
8     "createdAt": "2026-05-10T02:44:35.737547434Z"
9   },
10  "timestamp": "2026-05-10T02:44:35.751139850Z"
11 }
```

Add Favorite

Engagement Service – Create Review

Endpoint

POST /api/reviews

Features Demonstrated

- Reviews System
- Ratings System
- User Feedback Management

Library Management System - Render APIs / 08 - Engagement / Reviews / Create Review

POST `{{baseUrl}}/api/reviews` Send

Docs Params Authorization Headers (11) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (22) Test Results 201 Created 367 ms 901 B Save Response

`{}` JSON Preview Visualize

```

1  {
2    "success": true,
3    "message": "Review created successfully",
4    "data": {
5      "id": "f193a258-6253-4e9a-8ad5-34039ea8c7a8",
6      "userId": "8236eb27-4c7a-4407-b8df-4dcf2df20673",
7      "bookId": "aaaaaaaa-aaaa-aaaa-aaaaaaaaaaaa",
8      "rating": 5,
9      "comment": "Great book",
10     "createdAt": "2026-05-10T02:45:10.676604039Z",
11     "updatedAt": "2026-05-10T02:45:10.676604039Z"
12   },
13   "timestamp": "2026-05-10T02:45:10.600877158Z"
14 }

```

Create Review

2. Object Constraint Language (OCL)

The project applies business constraints and validation rules similar to OCL concepts.

Example Constraints

```

context User
inv UniqueEmail:
User.allInstances()->isUnique(email)

```

```

context Borrow
inv BorrowAvailableBook:
book.availableCopies > 0

```

```

context Review
inv ValidRating:
rating >= 1 and rating <= 5

```

```

context Book
inv PositiveCopies:
totalCopies > 0

```

OCL Validation from Real Code

The following screenshot demonstrates validation and business rules implemented inside the service layer.

Constraints Applied

- Email normalization
 - Duplicate email prevention
 - Transaction management
 - Business validation
-

3. Aspect Oriented Programming (AOP)

The project uses Aspect-Oriented Programming concepts for handling cross-cutting concerns.

AOP Features Used

- Logging
 - Monitoring
 - Exception Tracking
 - Proxy-Based Interception
-

AOP Concept

Instead of repeating logging code inside every service method:

```
System.out.println("Method Started");
```

Spring AOP intercepts methods automatically using proxies and aspects.

Benefits

- Separation of Concerns
 - Cleaner Code
 - Reusability
 - Centralized Logging
-

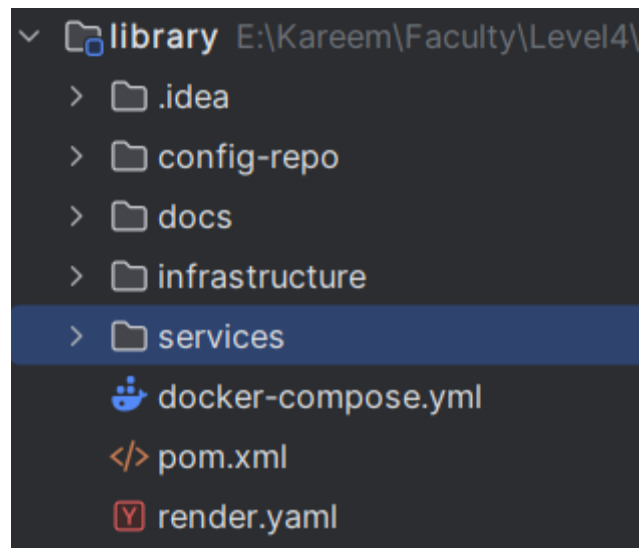
4. Docker

The entire system is dockerized.

Dockerized Components

- PostgreSQL
- Config Server
- Discovery Server
- API Gateway
- All Microservices

Project Infrastructure Structure



Project Structure

Running Docker Containers

The following screenshot demonstrates all running containers successfully.

☐	▼	●	library	-	-	-	1450.32%	2 seconds ago			
☐		●	catalog-service	ddf18a976b4e	library-catalog-service		211.7%	4 seconds ago			
☐		●	report-service	435161b52212	library-report-service		224.23%	4 seconds ago			
☐		●	engagement-service	e8b8f11da0d0	library-engagement-service		212.52%	5 seconds ago			
☐		●	circulation-service	a10702ae042d	library-circulation-service		212.31%	3 seconds ago			
☐		●	auth-service	fb9b1df7199a	library-auth-service		191.96%	3 seconds ago			
☐		●	api-gateway	a29ab7562c01	library-api-gateway	8080:8080	163.11%	6 seconds ago			
☐		●	discovery-server	b1aab3434400	library-discovery-server	8761:8761	176.02%	2 seconds ago			
☐		●	config-server	31cb38e3e06b	library-config-server	8888:8888	58.29%	13 seconds ago			
☐		●	postgres	f19be1cce03	postgres:16-alpine	5432:5432	0.18%	24 seconds ago			

Docker Containers

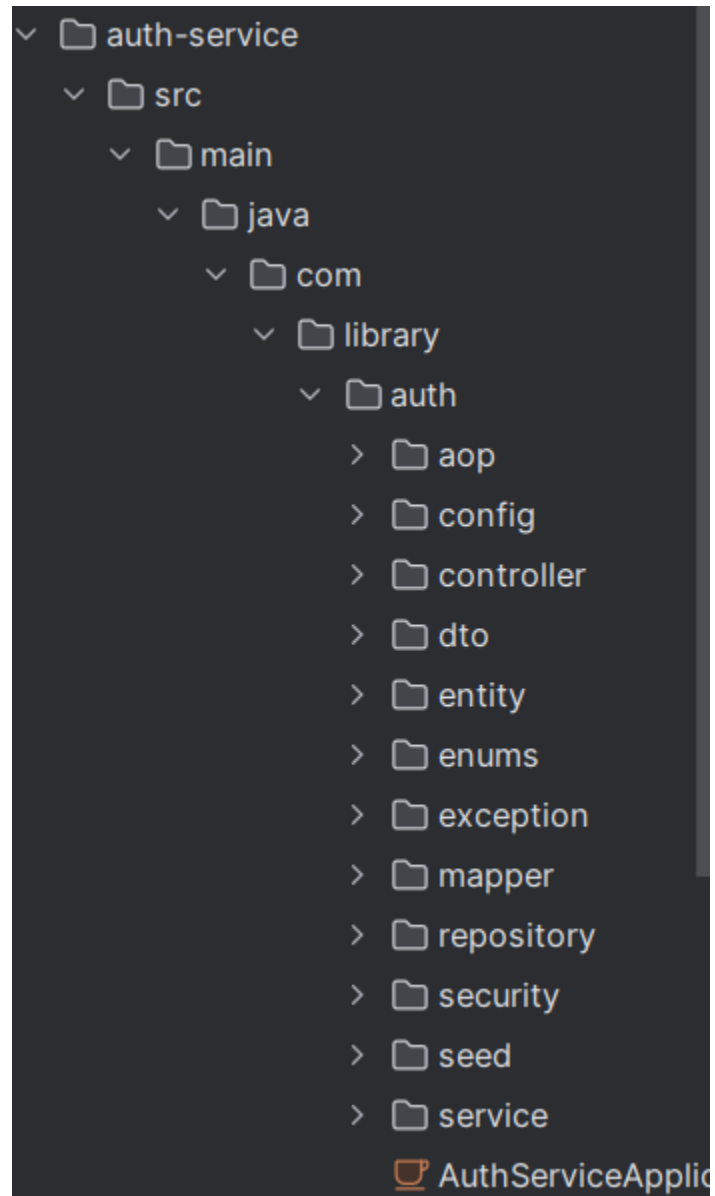
5. Clean Code

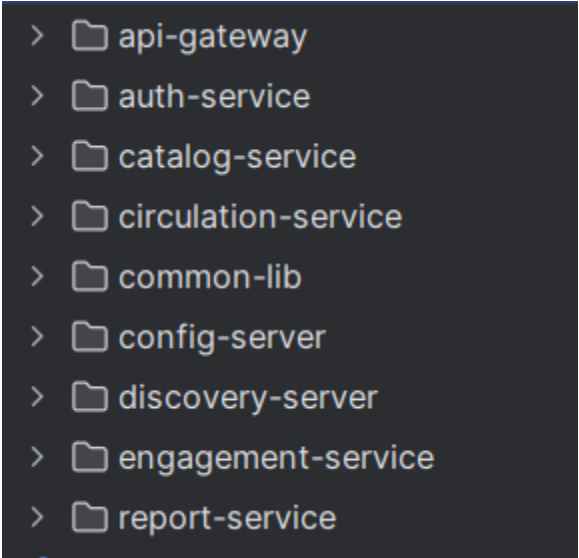
The project follows clean architecture and clean code principles.

Package Organization

- controller
 - service
 - repository
 - dto
 - mapper
 - config
 - security
 - exception
-

Organized Project Structure





```
> api-gateway
> auth-service
> catalog-service
> circulation-service
> common-lib
> config-server
> discovery-server
> engagement-service
> report-service
```

Clean Architecture

Clean Code Principles Applied

Separation of Concerns

Each layer has a clear responsibility.

Constructor Injection

Dependencies are injected using constructors.

DTO Pattern

DTOs are separated from entities.

Naming Conventions

Meaningful names are used for methods and classes.

6. Design Patterns

The project implements multiple design patterns.

1. Singleton Pattern

Usage

Spring Beans are singleton by default.

Code Example

```
@Service
public class AuthServiceImpl implements AuthService
```

Benefit

- Shared instance
 - Better memory management
 - Centralized service logic
-

2. Repository Pattern

Usage

Repositories abstract database access.

Code Example

```
public interface UserRepository extends JpaRepository<User, UUID>
```

Benefit

- Cleaner data access
 - Easier maintenance
 - Better abstraction
-

3. DTO Pattern

Usage

DTOs are used between APIs and services.

Examples

- LoginRequest
- RegisterRequest
- AuthResponse

Benefit

- Secure API contracts

- Prevent exposing entities
-

4. Builder Pattern

Usage

Objects are created using builders.

Example

`AuthResponse.builder()`

Benefit

- Cleaner object creation
 - Readable code
-

5. Proxy Pattern (AOP)

Usage

Spring AOP creates proxy objects around services.

Benefit

- Logging without modifying business logic
 - Centralized monitoring
-

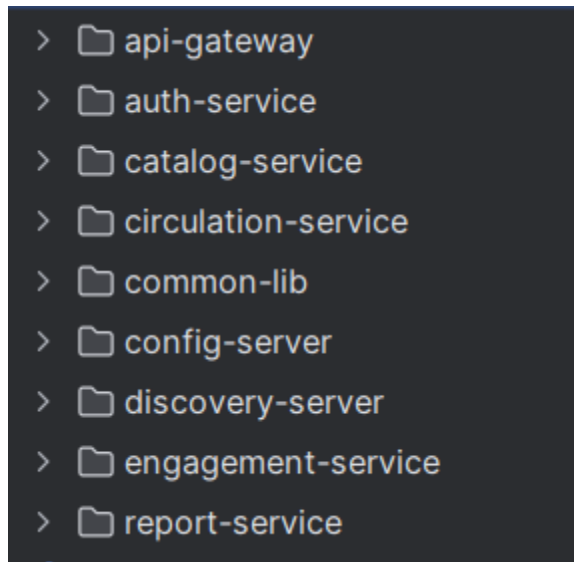
7. Microservices & Cloud

The project is fully implemented using Microservices Architecture.

Implemented Services

- API Gateway
 - Config Server
 - Discovery Server
 - Auth Service
 - Catalog Service
 - Circulation Service
 - Engagement Service
 - Report Service
-

Services Structure



Microservices Structure

Eureka Discovery Server

The following screenshot demonstrates all services registered successfully in Eureka.

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - a29ab7562c01:api-gateway:8080
AUTH-SERVICE	n/a (1)	(1)	UP (1) - fb9b1df7199a:auth-service:8081
CATALOG-SERVICE	n/a (1)	(1)	UP (1) - ddf18a976b4e:catalog-service:8082
CIRCULATION-SERVICE	n/a (1)	(1)	UP (1) - a10702ae042d:circulation-service:8083
ENGAGEMENT-SERVICE	n/a (1)	(1)	UP (1) - e8b8f11da0d0:engagement-service:8084
REPORT-SERVICE	n/a (1)	(1)	UP (1) - 435161b52212:report-service:8085

Eureka Dashboard

Cloud Deployment

The following screenshot demonstrates successful deployment and running services.

SERVICE NAME 9	STATUS	RUNTIME	REGION	UPDATED ↓
library-circulation-service ⌵	✓ Deployed	Docker	Oregon	29min ...
library-api-gateway ⌵	✓ Deployed	Docker	Oregon	50min ...
library-auth-service ⌵	✓ Deployed	Docker	Oregon	5h ...
library-engagement-service ⌵	✓ Deployed	Docker	Oregon	5h ...
library-catalog-service ⌵	✓ Deployed	Docker	Oregon	5h ...
library-report-service ⌵	✓ Deployed	Docker	Oregon	5h ...
library-discovery-server ⌵	✓ Deployed	Docker	Oregon	5h ...
library-config-server ⌵	✓ Deployed	Docker	Oregon	5h ...
library-postgres	✓ Available	PostgreSQL 18	Oregon	21h ...

Cloud Deployment

Conclusion

The Library Management System successfully demonstrates:

- RESTful APIs
- JWT Authentication
- Role-Based Access Control
- Dockerized Infrastructure
- Eureka Service Discovery
- API Gateway Routing
- Clean Architecture
- Design Patterns
- AOP Concepts
- OCL-style Constraints
- Cloud Deployment

The project is production-ready and fully deployed using Docker containers and cloud infrastructure.